

Explainability and Traceability of AI Systems through Provenance Tagging

Anonymous

No Institute Given

Abstract. As artificial intelligence (AI) systems evolve into complex, multi-stage workflows orchestrating large language models (LLMs) and dynamic code execution, their inherent non-determinism and opacity pose significant barriers to trust, transparency, and accountability. Current monitoring tools are often framework-dependent and primarily focus on tracing individual agent actions rather than providing a holistic view of interactions across multiple components. To address this challenge, we present a novel hybrid framework for generating traceable, on-demand explanations for AI systems. Our approach leverages the ProvONE and Explanation Ontologies to capture execution traces as a knowledge graph, which is then used to generate natural-language explanations for user questions through a KG-based retrieval mechanism. Departing from traditional open-ended retrieval, we leverage the ontology to systematically generate a set of SPARQL queries that act as retrieval functions. Given a user question, the system iteratively selects and executes appropriate queries over the KG, using retrieved information to refine subsequent steps until a satisfactory answer is obtained. The final results are aggregated as a structured chain of evidence to produce context-aware, verifiable explanations. We evaluate the efficacy of our approach through its implementation in ChatBS-NexGen, a dynamic LLM unit-testing system, and Biomni, a general-purpose biomedical AI agent. Our results on ChatBS-NexGen and Biomni show that this approach outperforms all baselines, achieving the highest semantic and grounding metrics and over 90% win rates in pairwise explanation comparisons.

Keywords: Explainable AI (xAI) · Provenance · AI Workflows · Knowledge graph-based xAI.

1 Introduction

Modern artificial intelligence (AI) systems have moved beyond simple input-output models to become sophisticated orchestration engines, shifting from providing static responses to performing dynamic program synthesis and agentic workflows [36]. Such workflows often orchestrate multiple invocations of Large Language Models (LLMs), which are used not only for text generation but also for synthesizing executable code, defining intermediate functions on-the-fly to fulfill user intents, and interacting with external environments and tools [36,12,16]. In this work, an “execution trace” refers to the runtime record of such a workflow: the

sequence of LLM calls, generated code or functions, tool invocations, input and output artifacts, intermediate data, and dependencies between workflow steps. While such architectures significantly elevate system capabilities, they inherit the non-determinism and opacity of their underlying models. This “black box” characteristic presents a significant barrier to adoption in critical domains [18], as it fundamentally undermines trustworthiness, accountability, traceability, and the ability to debug or verify system behavior. As AI-driven systems become more autonomous, the need for transparent, structured, and verifiable explanations grounded in such traces becomes a critical requirement for reliable system deployment.

Existing observability and tracing frameworks (e.g., OpenTelemetry [20]) primarily operate at the level of low-level service spans and function calls, while LLM-specific tracing tools (e.g., AgentOps [2]) are often framework-dependent. As a result, current approaches lack native support for capturing semantically meaningful, cross-component provenance across multi-component AI systems, including agentic workflows. This operational opacity creates a dual challenge: how can execution traces be efficiently captured, and how can these traces be synthesized into coherent explanations?

On the information retrieval side, existing approaches such as Retrieval-Augmented Generation (RAG) [25,1] and SPARQL generation strategies [35,22] treat retrieval as an open-ended generation problem or an LLM-guided graph traversal problem. However, structured workflow traces require an expressive model to capture all the nuances that happened during execution; ignoring this structure may lead to inefficiencies, increased hallucination risk, and reduced interpretability.

Motivated by these insights, we present a hybrid neuro-symbolic framework that leverages ontology-grounded knowledge graphs to provide traceable, on-demand explanations for complex AI workflows. Our approach transforms execution traces into a semantically structured resource and formulates explanation generation as a process of executing verifiable queries over this resource. To achieve this, we introduce a cohesive three-stage methodology: (1) a semantic annotation layer that captures workflow execution using ontologies such as ProvONE [6] and the Explanation Ontology (EO) [7], as well as annotation libraries in Python and R to capture intermediary artifacts of the AI systems; (2) a schema-driven generator that derives questions paired with executable SPARQL queries, forming a constrained function library; and (3) a neuro-symbolic explanation module that iteratively retrieves evidence using this function library.

By constraining retrieval to executable, ontology-aligned operations, the system ensures that generated explanations remain grounded in the underlying execution data. We evaluated our approach on ChatBS-NexGen [12] (an open-source platform for empirically validating LLM outputs against authoritative knowledge graphs) and Biomni [16] (a general-purpose biomedical AI agent), and results show that we outperform against baselines across semantic quality, grounding, and task-specific accuracy, with pairwise LLM-judged win rates exceeding 90% in most comparisons.

2 Methodology

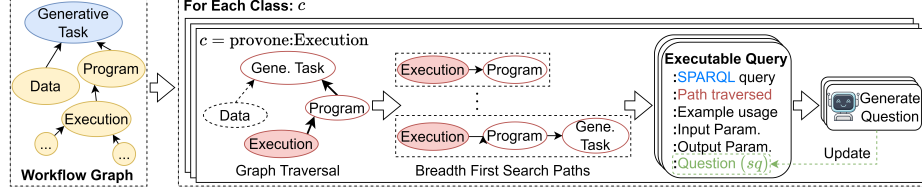


Fig. 1: Automated generation of the Synthetic Question library. The process involves (1) traversing the source ontology schema to find paths, (2) formulating executable SPARQL queries and input parameters for each path, (3) generating synthetic natural language questions using an AI agent, and (4) modeling the resulting query-question pairs as a structured graph.

Our methodology is designed to transform opaque AI workflow executions into queryable, provenance-grounded explanations for user questions about how a system behaved. It consists of two phases: a setup stage (Secs. 2.1 and 2.2) followed by a question-answering stage (Secs. 2.3 and 2.4). During setup, we add lightweight annotations to the workflow code to record provenance information during execution. To facilitate this, we developed a specialized Python/R library that abstracts the instantiation of ontology primitives (e.g., `provone:Program`, `eo:AITask`); however, the integration of these annotations into the codebase remains a developer-driven process, allowing for the targeted capture of program-specific logic and artifacts. The resulting workflow graph, together with the ontology schema, is then used to synthesize a set of synthetic questions (SQs) representing the set of questions that can be answered over the graph following a similar approach as [1]. Subsequently the generated SQs and workflow graph are used to retrieve context to answer user questions.

2.1 AI Workflow Representation and Workflow graph annotation

AI workflows are characterized by complex, nested, and interconnected processes, which makes modeling their execution a significant challenge. An AI component might produce an artifact or a function that is subsequently utilized in a different, separate workflow. To formally capture the elements of this execution, we reuse and align ProvONE [6](provone), the Explanation Ontology [7] (EO), the Semanticscience Integrated Ontology (SIO) [10], and W3C’s Provenance Ontology (PROV-O) [19] as depicted in Figure 2.

The ProvONE ontology provides the structural backbone for modeling the provenance and data flow of computational processes. We use its core classes to represent the “skeleton” and the runtime behavior of the workflow. For example:

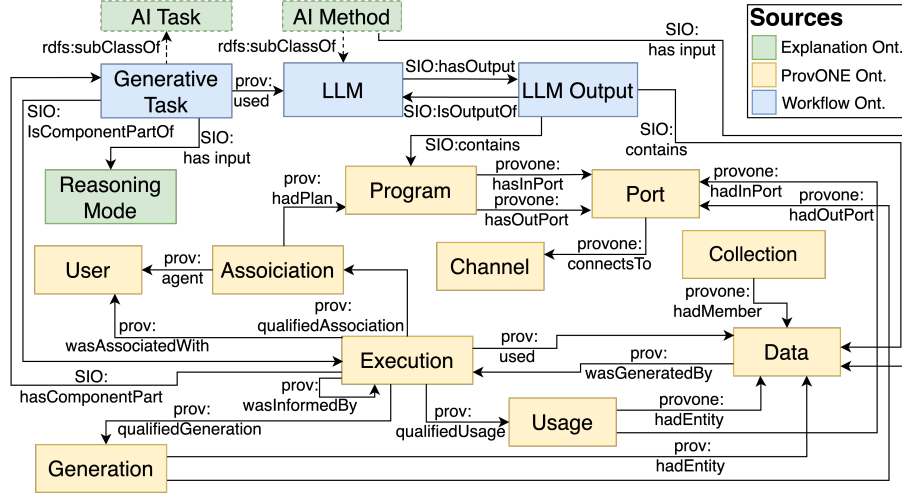


Fig. 2: Backing ontology for the workflow graph

- `provone:Program` represents the specification or definition of a particular workflow step.
- `provone:Execution` captures a specific runtime instance of that `Program`.
- `provone:Data`, `provone:Collection`, and `provone:Port` are used to model the data artifacts that are consumed and produced by a program during its execution.
- `provone:User` and `prov:Association` link the agents (users or other systems) responsible for initiating or designing the workflow.

While ProvONE excels at providing the concepts for modeling the structure of the execution trace (i.e., “what ran and what data was used”), it does not inherently describe the purpose or nature of the AI tasks themselves coupled with their inputs and outputs. EO provides a set of classes that can be used to semantically enrich the ProvONE model, allowing us to describe the “what” and “why” of the AI-driven steps. As shown in Figure 2, we extend EO to create subclasses to represent Generative Tasks, LLM-based AI Methods and specific outputs (`provone:Program` and `provone:Data`) that an LLM may generate.

- To model generated functions, we utilize `workflow:LLMOutput` $\xrightarrow{\text{sio:contains}}$ `provone:Program` connections.
- To represent the use of generative tasks within a function as a component, we utilize:
 - `provone:Execution` $\xrightarrow{\text{sio:hasComponentPart}}$ `workflow:Generative_Task`
 - `workflow:Generative_Task` $\xrightarrow{\text{sio:isComponentPartOf}}$ `provone:Execution`

This design allows us to capture structural provenance (captured by ProvONE and PROV-O) and task-level semantics (captured by EO and SIO), enabling both

execution traceability and explanation generation within a unified representation. The resulting schema defines the primitives and relations required to represent AI workflow executions, including programs, executions, ports, data artifacts, generated functions, and generative tasks. To operationalize this schema in real systems, we develop companion Python and R libraries that allow workflow code to be annotated and serialized into a workflow graph.

2.2 Python/R Library

The Python and R provenance libraries implement the annotation schema described above and support the construction of workflow graphs from AI system executions. The libraries provide high-level functions for declaring workflow programs, ports, channels, executions, inputs, outputs, and inter-step dependencies. Rather than requiring users to manually construct RDF triples, the API records each function or agent step as a `provone:Program` and each runtime invocation as a `provone:Execution`, linked to the artifacts it used or generated.

The libraries are designed to capture intermediate states that are often omitted from end-to-end workflow logs, including prompts, generated responses, extracted values, data-frame rows, lists, observations, and final outputs. These artifacts are represented as provenance data objects or collections and connected to the corresponding execution. For LLM-based workflows, the libraries also record model calls as generative tasks, including the model used, the inputs supplied, and the outputs produced. The captured annotations are then serialized as Turtle [4], producing the workflow graph used by the explanation system.

2.3 Synthetic Question creation

Since the generated workflow graph strictly adheres to a backing ontology (Fig. 2), we can formulate the information retrieval task as the execution of a predefined set of SPARQL queries, thereby transforming the problem from an open-ended generation task to a closed-set selection task. Our goal is to model a set of SPARQL queries that represent the questions that “can be answered” by this KG. We initiate this process by traversing the ontology to identify viable semantic paths, where we define a viable semantic path as any acyclic sequence of properties connecting a source class to a target class (e.g., `provone:Program` $\rightarrow$ `provone:hasOutPort` $\rightarrow$ `provone:Port`). Each path represents a distinct unit of information (e.g., “What output ports are associated with this program?”) that can be extracted from the workflow graph.

We define a *synthetic question* SQ as a tuple $SQ = (id, sq, Q_{sparql}, U_{sparql}, \mathcal{I}, \mathcal{O})$, where id is a unique identifier, sq is the natural language question, Q_{sparql} is the corresponding executable SPARQL query, U_{sparql} is an example usage of Q_{sparql} , and $\mathcal{I}$ and $\mathcal{O}$ denote the input and output specifications, respectively. The generation of these programs proceeds in three phases (Fig. 1) using a schema-guided crawler:

1. **Schema-Guided Traversal:** We employ a Breadth First Search (BFS) over the ontology schema (rather than instance data). Starting from a class C_{start} (e.g., `provone:Execution`), the algorithm traverses relationships r to identify valid semantic paths $p = \{C_{start} \xrightarrow{r_1} C_1 \dots \xrightarrow{r_n} C_n\}$. The traversal depth is bounded by a parameter LP_{max} (set to 5 in all experiments). This process is repeated for each class in the schema, and the set of unique paths is collected.
2. **Query Formulation:** For each identified path p , the system constructs parameterized SPARQL queries Q_{sparql} using templating. For every path, two queries are generated corresponding to forward and reverse traversal, enabling retrieval in both directions (i.e., from source to target and vice versa). In parallel, an LLM (using a 1-shot strategy; Fig. ??) generates the corresponding natural language question sq by interpreting the path semantics (e.g., “What activity generated this entity?”).
3. **Filtering:** To ensure that generated SQ s correspond to actual data, we perform a filtering step. Each candidate query is executed on the workflow graph using a sample instantiation U_{sparql} . Queries that return empty results are discarded, ensuring that only SQ s that contain entities (useful in the context of retrieval) in the workflow graph are retained.

In addition, we construct SQ s for common retrieval operations, including retrieving all objects of a given class, enumerating properties of a given object, and filtering objects of a class based on property values. All generated SQ s are used as a library of functions (the SQ Library), where each SQ represents an answerable question over the AI system. An example of an SQ is provided in Appendix C.

2.4 Explanation Question Answering System

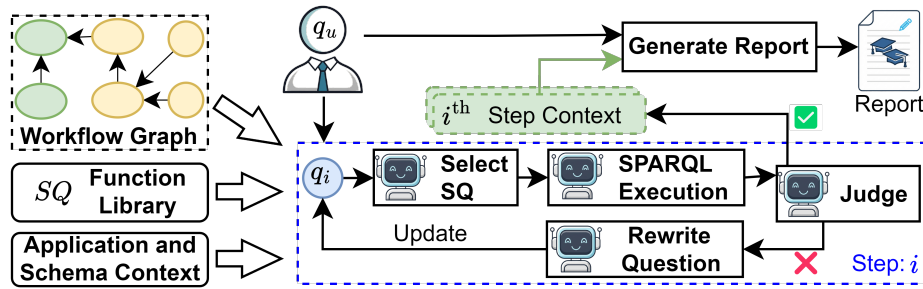


Fig. 3: Overview of the retrieval pipeline. Given a user question, the system iteratively selects an SQ , executes its Q_{sparql} , and evaluates the result. If insufficient, the question is updated and the process repeats. The accumulated context is used to generate the final report.

To effectively utilize the *SQ* library generated in Section 2.3, we develop a neuro-symbolic question-answering system that bridges the gap between unstructured natural-language queries and structured SPARQL-based retrieval over the workflow graph. As illustrated in Figure 3, the system leverages the *SQ* library as a constrained set of executable retrieval functions. Given a user question q_u , the system performs a sequence of retrieval steps. At step i , the system maintains a step question q_i , which represents the current information need to be resolved. The process begins with $q_0 = q_u$, and each subsequent step updates q_i using the information retrieved so far and feedback from the judge.

The retrieval process proceeds as an iterative loop:

- **SQ Selection:** Given the current user question, the system selects an appropriate synthetic question (SQ_i) from the *SQ* Library based on the available context. In the first loop, the *SQ* Selection is restricted to common retrieval operations.
- **SPARQL Execution:** The parameterized SPARQL query (Q_{sparql}) in the selected *SQ* is instantiated using $\mathcal{I}$ and q_i and executed over the workflow graph to retrieve relevant information/entities (e_i).
- **Answer Evaluation:** A summarization component utilizes the retrieved information of the previous and current steps to answer q_i , and the answer is denoted as a_i . The judge validation component (Judge) utilizes the retrieved results and a_i to determine whether they sufficiently answer the user question q_u . If not, the judge provides feedback on the missing information. If the provided context is sufficient to answer q_u , the judge provides a refined answer a_{final} to q_u and terminates the loop.
- **Question Update:** If the answer is insufficient, the system updates the step question for the next step (q_{i+1}) by incorporating the retrieved information and the judge feedback received, and proceeds to the next iteration.

This loop is executed for a fixed number of steps (set to 3 in our experiments) or until a satisfactory answer is obtained. If no answer is found, the system falls back to retrieving objects from a predefined set of classes. An example of stepwise execution for a question is provided in appendix Fig. 6. In the final phase, the system aggregates the results obtained for all steps to construct an explanation for the original query q_u . Let $\mathcal{C} = \{(q_i, SQ_i, a_i, e_i)\}_{i=1}^n$ denote the set of step records: step-question, executed *SQ*, intermediate answer, and evidence entity triplets. $\mathcal{C}$ is structured as a report denoting the chaining of step-by-step answering of step questions, adding a_{final} explanation at the end. An example report is provided in Fig. 5. This design ensures that each component of the final explanation is grounded in verifiable data retrieved from the workflow graph, reducing hallucination and improving interpretability.

3 Evaluation

The primary goal of our evaluation is to determine whether the system’s generated explanations are complete, factually grounded, and relevant to the user’s inquiry.

Complementing this qualitative assessment, we also quantitatively evaluate the retrieval mechanism by measuring the recall of key entities.

3.1 Implementation

The SQ creation pipeline and question answering pipeline are implemented using Python employing the DSPy framework [17] for programmatic prompting. Large language models are self-hosted on an NVIDIA DGX server and accessed via LM Studio. Across all methods, we use `gpt-oss-20b` for LLM calls and `text-embedding-bge-large-en-v1.5` for embedding creation.

3.2 Baselines

We evaluate our framework against baselines representing distinct paradigms for querying KGs.

GroundedWorkflowBaseline (GWB). A symbolic, grounding-first approach that answers queries using deterministic heuristics over the workflow graph. It classifies questions (e.g., model, input, lineage) and retrieves evidence using lexical matching and explicit relations (e.g., `prov:used`, `prov:wasGeneratedBy`). While efficient and interpretable, it is limited to predefined patterns and lacks multi-hop reasoning.

VectorSimilarityBaseline (VSB). A retrieval-augmented baseline that embeds KG-derived object descriptions (constructed from RDF subjects, labels, types, and outgoing properties) into a vector space. For each query, the top- k similar objects are retrieved and passed to an LLM for answer generation. While effective for semantic matching, it treats objects independently and does not preserve multi-hop structure, ontology semantics, or predicate directionality.

GRASP. A zero-shot KGQA method that performs iterative SPARQL generation via LLM-guided function calls, refining queries using intermediate feedback [35]. While expressive, it requires dynamic query construction at inference time, introducing latency and sensitivity to entity and query formulation errors.

HippoRAG-KG (HippoRAG). A graph-based RAG baseline adapted from HippoRAG [15]. The KG is transformed into object-level documents and OpenIE-style triples, enabling graph-based retrieval without LLM-based extraction. While this preserves the relational structure, RDF semantics, such as datatype constraints and predicate identity, are flattened into textual representations.

HyperGraphRAG-KG (HyperGRAG). A hypergraph-based RAG baseline adapted from HyperGraphRAG [24]. The KG is converted into entities and hyperedge-like fragments, enabling associative retrieval over structured data. However, this transformation loses RDF-specific semantics such as directionality, datatypes, and ontology constraints.

These baselines span symbolic retrieval, vector-based retrieval, and iterative query synthesis. In contrast, our approach precomputes ontology-grounded *SQs*, eliminating query generation at inference time and enabling structured, traceable retrieval over KG. Appendix F provides the full adaptation protocol including exact representation exposed to each baseline

3.3 Evaluation Question Construction

Table 1: A few evaluation question examples categorized by question type.

Question	Category
In what places do we utilize AI in this workflow?	<i>Entity</i>
What are the inputs used by the large language model to create the “Extract entities” function?	<i>Entity</i>
What is the number of outputs for the “Output port for kg_info” in the execution <http://testwebsite/testProgram#id_20260420105727_300>?	<i>Numeric</i>
Did the execution <http://testwebsite/testProgram#id_20260420105710_61> use data that were created by a generative task?	<i>True/False</i>
Do any executions of the “Generates a system prompt” function include a generative task as a component?	<i>True/False</i>
What are the outputs of the generative task that was a component in executions of the “Chat with the LLM” function?	<i>Entity</i>

To rigorously assess the interpretability and retrieval accuracy of our framework, we curated a benchmark dataset consisting of manually annotated natural language queries. Each dataset entry is formally defined as a tuple $T = (Q, E_{gt}, Q_{sparql}, S_{ent})$, where:

- Q is the natural language question derived from templated patterns to ensure linguistic consistency while maximizing coverage.
- E_{gt} is the ground truth textual explanation that represents the ideal response expected from the system.
- Q_{sparql} is the corresponding SPARQL query that retrieves the information required to construct E_{gt} .
- S_{ent} is the set of critical knowledge graph entities (URIs and literals) that must be present in a well-grounded explanation. These entities serve as a grounding constraint, ensuring that generated responses are anchored to the correct graph nodes.

We categorize the evaluation questions into three types, as shown in Table 1. *Entity* questions require retrieving specific entities or sets of entities associated with a given concept or execution. *True/False* questions evaluate the system’s ability to verify the presence or absence of specific conditions within the workflow. *Numeric* questions assess the system’s completeness in retrieval by requiring aggregation over all relevant instances, thereby testing whether the system captures the full extent of the underlying data rather than partial results.

3.4 Evaluation Metrics

To evaluate system performance, we employ an automated LLM-as-a-judge framework that enables scalable assessment against ground-truth annotations. Given a user question Q , ground truth explanation E_{gt} , and model-generated response E_{pred} , an evaluator LLM assigns normalized scores in the range $[0, 1]$ across three dimensions:

1. **Completeness (Comp.):** The extent to which E_{pred} covers the full information content of E_{gt} . A response is considered complete only if all key semantic components present in the ground truth are captured.
2. **Faithfulness (Faith.):** The degree to which E_{pred} remains faithful to E_{gt} , penalizing hallucinations, unsupported claims, or contradictions. This metric replaces traditional accuracy by explicitly measuring grounding to the reference explanation.
3. **Relevance (Rel.):** The degree to which the response directly addresses the user question Q , penalizing inclusion of extraneous or off-topic information.

We acknowledge that LLM-as-a-judge evaluation may introduce evaluator bias and may not fully capture whether a response is grounded in the correct KG entities. To address this, we complement semantic scoring with entity-recall metrics and exact-answer accuracy for question types with definitive answers. For entity grounding, we compute:

$$R_{final} = \frac{|S_{ent} \cap S_{ret_final}|}{|S_{ent}|}, R_{total} = \frac{|S_{ent} \cap S_{ret_total}|}{|S_{ent}|}$$

where S_{ret_final} denotes entities retrieved in the final step of the execution plan, and S_{ret_total} denotes entities retrieved across all steps. For *True/False* and *Numeric* questions, we compute exact-answer accuracy by comparing them with the ground truth as they have definitive answers. We denote the accuracies as Acc_{bool} and $Acc_{numeric}$.

In addition to metric-based evaluation, following [37], we assess the understandability of generated explanations through a pairwise win-rate analysis. Using an automated LLM-as-a-judge framework, the evaluator is presented with two candidate explanations and selects the more understandable and faithful response with respect to the ground truth. We compute win rates as the proportion of pairwise comparisons in which a given method is preferred over competing baselines, with ties counted as half wins. For both absolute scoring and pairwise evaluation, we employ **LLaMA-3.3-70B-Instruct** (hosted via LM Studio) as the evaluation model. The prompts used for these evaluations are provided in Appendix E.

3.5 ChatBS-NexGen Explanation Module

To evaluate our framework, we integrate it with ChatBS-NexGen [12], an open-source platform for validating LLM outputs against authoritative knowledge graphs. ChatBS-NexGen supports controlled LLM experimentation through

prompt configuration, repeated model submissions, response logging, and KG-based validation. We focus on its dietary recommendation validation task, where personalized prompts are generated from patient attributes (e.g., diagnosis, health indicators, food preferences, and allergies) and LLM-generated food recommendations are validated against DBpedia [3] and related ontologies. For example, the original study examined whether recommended foods were appropriate for diabetic patients by linking food mentions to glycemic index information, exposing cases where LLMs suggested high-glycemic-index foods to diabetic users. This use case is a suitable testbed because it combines dynamically generated functions, repeated LLM calls, KG-based validation, and downstream steps where intermediate outputs influence later processing. We extend ChatBS-NexGen by instrumenting its execution pipeline with our R library, enabling automatic capture of execution traces and generation of structured, provenance grounded explanations.

We constructed a benchmark comprising 103 questions, each paired with a manually authored SPARQL query to retrieve the corresponding ground-truth answers from the ChatBS-NexGen workflow graph. The dataset is distributed across the following categories: 38 True/False, 49 Entity, and 16 Numeric questions. These questions were used to generate explanation responses from each system under evaluation, and the aggregated results are reported in Table 2.

Table 2: Mean evaluation results for ChatBS-NexGen [12]. Metrics report completeness (Comp.), faithfulness (Faith.), relevance (Rel.), final and total entity recall (R_{final} , R_{total}), and boolean/numeric accuracy (Acc_{bool} , $Acc_{numeric}$). Scores are percentages and dashes indicate metrics not available for the corresponding method.

Method	Comp.	Faith.	Rel.	R_{final}	R_{total}	Acc_{bool}	$Acc_{numeric}$
GWB	14.9	9.9	34.9	23.36	35.51	0.0	12.5
VSB	22.0	27.2	61.9	9.61	14.49	34.2	6.2
GRASP	50.2	51.2	69.7	2.7	2.7	28.9	12.5
HippoRAG	27.7	28.0	62.5	34.5	57.26	52.6	25.0
HyperGRAG	34.0	36.3	64.4	-	-	42.1	25.0
Ours	64.5	70.2	89.5	50.61	62.12	63.2	50.0

3.6 Biomni Explanation module

For our second task, we integrate our framework with Biomni [16], a general-purpose biomedical AI agent that combines LLM-based reasoning, planning, and code execution. Biomni is a suitable testbed because it dynamically selects tools, executes code, and uses intermediate observations to guide subsequent actions, producing complex execution traces across biomedical resources. We evaluate our explanation module on a Biomni execution for the user request: “Predict ADMET

properties for this compound: CC(C)CC1=CC=C(C=C1)C(C)C(=O)O.” The agent was run using `gpt-oss-20b` with access to a BioConda-based environment containing biomedical Python, R, and command-line tools, including chemistry and drug-discovery libraries such as RDKit, DeepPurpose, OpenBabel, and PyTDC. We annotate Biomni’s execution loop using our Python library, serializing intermediate artifacts, tool invocations, LLM calls, generated code, and outputs into a structured workflow graph. This enables provenance-grounded explanations of how the agent selected tools, produced intermediate results, and arrived at the final ADMET prediction.

We constructed a benchmark comprising 106 questions, each paired with a manually authored SPARQL query to retrieve the corresponding ground-truth answers from the Biomni workflow graph. The dataset is distributed across the following categories: 39 True/False, 46 Entity, and 21 Numeric questions. These questions were used to generate explanation responses from each system under evaluation, and the aggregated results are reported in Table 3.

Table 3: Mean evaluation results for Biomni [16]. Metrics report completeness (Comp.), faithfulness (Faith.), relevance (Rel.), final and total entity recall (R_{final} , R_{total}), and boolean/numeric accuracy (Acc_{bool} , $Acc_{numeric}$). Scores are percentages and dashes indicate metrics not available for the corresponding method.

Method	Comp.	Faith.	Rel.	R_{final}	R_{total}	Acc_{bool}	$Acc_{numeric}$
GWB	11.89	6.98	34.06	33.6	36.73	0.0	14.29
VSF	22.90	22.84	61.72	5.64	15.58	15.73	0.0
GRASP	46.89	46.42	73.35	2.27	2.27	38.46	19.05
HippoRAG	26.27	31.89	68.63	39.14	66.58	48.79	33.33
HyperGRAG	49.56	48.76	76.27	-	-	53.85	28.57
Ours	65.66	66.48	91.36	46.81	55.68	76.92	52.38

Comparative Study. Across both benchmarks, our approach consistently outperforms all baselines on the primary evaluation metrics. On Biomni, our method achieves the highest scores in completeness, faithfulness, and relevance, indicating that the generated explanations more closely align with the ground truth while remaining accurate and focused on the user question.

A similar trend is observed on ChatBS-NexGen, where our method again attains the highest scores in completeness, faithfulness, and relevance, along with the best performance on both boolean and numeric questions. While some baselines achieve competitive total retrieval scores, they do not translate this into high-quality explanations, as reflected in their lower semantic alignment metrics. In contrast, our approach maintains strong performance across both retrieval and explanation quality, highlighting its ability to produce more accurate and well-grounded responses.

3.7 Win Rate Analysis

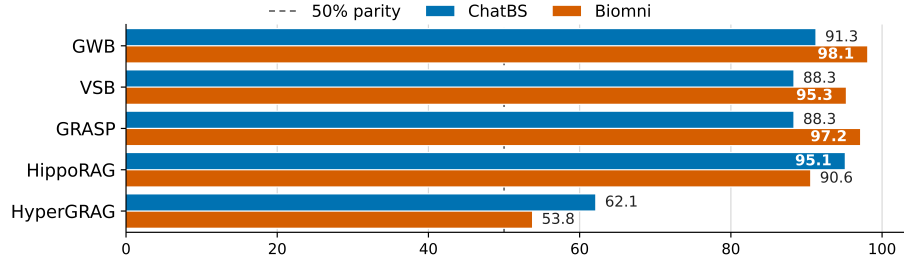


Fig. 4: Pairwise answer win rates of ours against each baseline on the ChatBS and Biomni evaluations. Values show the percentage of pairwise comparisons won by our method. The dashed line marks the 50% parity threshold.

As shown in Fig. 4, the pairwise win-rate results further highlight the effectiveness of our approach across both datasets. On ChatBS-NexGen, our method outperforms all baselines, achieving win rates above 88% against GWB, VSB, GRASP, and HippoRAG. The comparison with HyperGRAG is lower at 62.1%, but still remains above the 50% parity threshold, indicating that our explanations are preferred more often than those generated by the baseline. On Biomni, our method shows similarly strong performance, with win rates exceeding 95% against GWB, VSB, and GRASP, and 90.6% against HippoRAG. The comparison with HyperGRAG is closer than in ChatBS-NexGen, with a win rate of 53.8%, but it still remains above parity. This suggests that HyperGRAG is a stronger competitor in the Biomni setting, likely because its hypergraph representation can retrieve related entities and facts together, while our approach still maintains a slight preference advantage.

Overall, these results demonstrate that our approach consistently produces preferable explanations compared to all baselines, with particularly strong gains over symbolic and query-generation methods and robust improvements across both datasets.

4 Related Work

Our work makes contributions in two primary domains: the modeling of provenance in AI workflows and advancements in retrieval-augmented generation (RAG) for knowledge graphs.

4.1 AI Workflow Modeling & Provenance

Provenance tracking is widely recognized as essential for ensuring transparency, accountability, and explainability in machine learning (ML) pipelines [29,31].

While seminal surveys [29] argue that provenance is critical for trustworthy AI, they note that many current implementations remain ad-hoc. To address this, frameworks like MLflow2PROV [31] extract end-to-end provenance from industry-standard tools (e.g., Git, MLflow), validating the use of the PROV-O ontology [19] as a robust backbone for serializing ML pipelines into Knowledge Graphs (KGs). Similarly, recent adaptations [27] have extended PROV-O to create the Safe Haven Provenance ontology, automating tracking within Trusted Research Environments.

Recent work has increasingly linked provenance with Explainable AI (XAI). The concept of “Data Explainability” [26] suggests that traditional feature-attribution methods (e.g., SHAP [23], LIME [30]) must be complemented by pre-processing provenance to establish reliability. Approaches such as xMML-PPP [26] link retrospective provenance with model explainability results, while others [34] leverage provenance metadata to quantify specific metrics for transparency and accountability. Furthermore, the Data Journey Ontology (DJO) [9] has been utilized to extract multi-layered abstract graphs from Python notebooks to categorize training activities. While these works effectively model static or linear training pipelines, our work specializes in programmatic, compositional workflows containing non deterministic components (AI agents). We extend the abstraction capability of prior art by employing flexible synthetic questions to model user intents, moving beyond static graph queries.

4.2 Ontology-Guided KG-RAG & SPARQL generation

Recent advancements in KG question answering (KGQA) and KG-based RAG increasingly combine LLMs with structured retrieval. Systems such as byokg-rag [25] use multi-strategy retrieval to ground answers in graph derived entities, while SimGRAG [5] aligns user questions with graph patterns using Graph Semantic Distance. Self-supervised approaches such as BYOKG [1] further employ symbolic agents to explore KGs and synthesize query-program exemplars for zero-shot generalization. More recent structure-aware RAG methods, including HippoRAG [14] and HippoRAG2 [15], construct graph structures from text and use graph traversal mechanisms such as personalized PageRank to support multi-hop and associative retrieval. While effective, these approaches typically rely on induced graph structures, extracted triples, embeddings, and ranking heuristics, making retrieval dependent on extraction and linking quality.

A closely related line of work focuses on translating natural language questions into executable SPARQL queries. LLM-based text-to-SPARQL methods enable zero-shot and few-shot query construction, with systems such as SPINACH [22] iteratively exploring Wikidata through specialized functions and GRASP [35] generalizing this strategy across RDF knowledge graphs. Other recent systems use retrieval-augmented prompting and schema-aware context injection to generate SPARQL over domain-specific or federated KGs [11,32]. However, generated SPARQL can suffer from structural and semantic errors, including incorrect triple patterns and entity mismatches [28], and recent analyses suggest that some performance gains may reflect memorization rather than robust generalization [13].

Despite this progress, the use of ontologies to constrain retrieval remains under-explored. Some methods employ ontologies for path pruning [21], and tools such as Q2Forge [33] and RevOnt [8] partially automate competency question generation through user supervision or data-driven abstraction. In contrast, our approach uses the ontology schema directly to enumerate valid query paths and precompute question-SPARQL pairs that the ontology-backed KG can answer. This shifts retrieval from open-ended graph traversal or query generation to selection over a validated query set, improving interpretability, controllability, and faithfulness of the generated explanations.

5 Conclusion and Future Work

We presented a hybrid framework that addresses the opacity of multi-stage AI systems by integrating semantic knowledge representation with structured explanation generation. By grounding execution traces in ProvONE and EO, our approach reformulates explanation generation as a constrained retrieval process over a pre-constructed library of executable SPARQL functions. This enables explanations that are not only fluent, but also verifiably grounded in provenance data. We demonstrated the effectiveness of the framework through its integration with ChatBS-NexGen [12] and Biomni [16]. Across both systems, our approach achieved the strongest overall performance, obtaining the highest completeness, faithfulness, and relevance scores, as well as the best boolean and numeric accuracy. Pairwise win-rate analysis further showed that our explanations were preferred over all baselines, with win rates consistently above the 50% parity threshold and exceeding 90% in most comparisons. These results indicate that the proposed framework improves both the factual grounding and practical usefulness of explanations for dynamically generated AI workflows.

Despite these advantages, several limitations remain. The framework relies on developer-driven provenance instrumentation, which introduces engineering overhead. The multi-step inference pipeline also incurs latency due to sequential LLM calls, limiting real-time applicability. In addition, explanation completeness depends on the coverage of the SQ Library, requiring fallback strategies for out-of-scope questions. Finally, while our evaluation covers correctness, grounding, and traceability, further user-centered evaluation is needed to assess interpretability and practical utility. Future work will address these limitations by improving developer support for provenance instrumentation, reducing latency through parallelization and lightweight models, and expanding SQ Library coverage through adaptive generation. Overall, this work demonstrates that transforming execution traces into ontology-grounded, executable retrieval structures provides a practical path toward more transparent, verifiable, and accountable AI systems.

Supplemental Material Statement: Source code for our system, baselines (some available through sub modules), benchmarks and raw results are available at <https://anonymous.4open.science/r/LLM-Workflow-Explorer-040D>

Declaration on Generative AI

During the preparation of this work, the author(s) used GPT-5 in order to: Grammar and spelling check. After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content.

References

1. Agarwal, D., Das, R., Khosla, S., Gangadharaiyah, R.: Bring Your Own KG: Self-Supervised Program Synthesis for Zero-Shot KGQA. In: Duh, K., Gomez, H., Bethard, S. (eds.) Findings of the Association for Computational Linguistics: NAACL 2024. pp. 896–919. Association for Computational Linguistics, Mexico City, Mexico (Jun 2024). <https://doi.org/10.18653/v1/2024.findings-naacl.57>
2. AgentOps-AI: Agentops: Python sdk for ai agent monitoring, llm cost tracking, benchmarking, and more. <https://github.com/AgentOps-AI/agentops> (2025), gitHub repository; latest release listed as 0.4.21 on August 29, 2025; accessed May 7, 2026
3. Auer, S., Bizer, C., Kobilarov, G., Lehmann, J., Cyganiak, R., Ives, Z.: DBpedia: A Nucleus for a Web of Open Data. In: The Semantic Web. pp. 722–735. Springer (2007)
4. Beckett, D., Berners-Lee, T., Prud'hommeaux, E., Carothers, G.: RDF 1.1 turtle: Terse RDF triple language. W3c recommendation, World Wide Web Consortium (W3C) (Feb 2014), <http://www.w3.org/TR/2014/REC-turtle-20140225/>, latest version available at <http://www.w3.org/TR/turtle/>
5. Cai, Y., Guo, Z., Pei, Y., Bian, W., Zheng, W.: SimGRAG: Leveraging similar subgraphs for knowledge graphs driven retrieval-augmented generation. In: Che, W., Nabende, J., Shutova, E., Pilehvar, M.T. (eds.) Findings of the Association for Computational Linguistics: ACL 2025. pp. 3139–3158. Association for Computational Linguistics, Vienna, Austria (Jul 2025). <https://doi.org/10.18653/v1/2025.findings-acl.163>, <https://aclanthology.org/2025.findings-acl.163/>
6. Cao, Y., Jones, C., Cuevas, V., Jones, M., Ludäscher, B., McPhillips, T., Missier, P., Schwalm, C., Slaughter, P., Vieglais, D., Walker, L., Wei, Y.: ProvONE: extending PROV to support the DataONE scientific community (06 2016)
7. Chari, S., Seneviratne, O., Gruen, D.M., Foreman, M.A., Das, A.K., McGuinness, D.L.: Explanation ontology: A model of explanations for user-centered ai (2020), <https://arxiv.org/abs/2010.01479>
8. Ciroku, F., De Berardinis, J., Kim, J., Meroño-Peñuela, A., Presutti, V., Simperl, E.: RevOnt: Reverse engineering of competency questions from knowledge graphs via language models **82**, 100822. <https://doi.org/10.1016/j.websem.2024.100822>, <https://linkinghub.elsevier.com/retrieve/pii/S1570826824000088>
9. Daga, E., Groth, P.: Data journeys: Explaining AI workflows through abstraction. Semantic Web **15**(4), 1057–1083 (2024). <https://doi.org/10.3233/SW-233407>, <https://journals.sagepub.com/doi/abs/10.3233/SW-233407>
10. Dumontier, M., Baker, C.J., Baran, J., Callahan, A., Chepelev, L., Cruz-Toledo, J., Del Rio, N.R., Duck, G., Furlong, L.I., Keath, N., Klassen, D., McCusker, J.P., Queralt-Rosinach, N., Samwald, M., Villanueva-Rosales, N., Wilkinson, M.D., Hoehndorf, R.: The SemanticScience Integrated Ontology (SIO) for biomedical research and knowledge discovery. Journal of Biomedical Semantics **5**(1), 14 (2014). <https://doi.org/10.1186/2041-1480-5-14>, <http://sio.semanticscience.org>

11. Emonet, V., Bolleman, J., Duvaud, S., Mendes de Farias, T., Sima, A.C.: LLM-based SPARQL query generation from natural language over federated knowledge graphs. In: ISWC 2024 Special Session on Harmonising Generative AI and Semantic Web Technologies, November 13, 2024, Baltimore, Maryland. CEUR Workshop Proceedings, vol. 3953 (2025), cEUR-WS.org, online <https://ceur-ws.org/Vol-3953/355.pdf>
12. Erickson, J.S., Santos, H., Pinheiro, V., McCusker, J.P., McGuinness, D.L.: LLM experimentation through knowledge graphs: Towards improved management, repeatability, and verification. *Journal of Web Semantics* **85**, 100853 (2025)
13. Gashkov, A., Perevalov, A., Eltsova, M., Both, A.: SPARQL query generation with LLMs: Measuring the impact of training data memorization and knowledge injection. In: Web Engineering: 25th International Conference, ICWE 2025, Delft, Netherlands, June 30–July 3, 2025, Proceedings (2025), winner of Best Paper Award at the 25th International Conference on Web Engineering (ICWE 2025)
14. Gutiérrez, B.J., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: Hipporag: Neurobiologically inspired long-term memory for large language models. In: The Thirty-eighth Annual Conference on Neural Information Processing Systems (2024), <https://openreview.net/forum?id=hkujvAPVsg>
15. Gutiérrez, B.J., Shu, Y., Qi, W., Zhou, S., Su, Y.: From rag to memory: Non-parametric continual learning for large language models (2025), <https://arxiv.org/abs/2502.14802>
16. Huang, K., Zhang, S., Wang, H., Qu, Y., Lu, Y., Roohani, Y., Li, R., Qiu, L., Li, G., Zhang, J., Yin, D., Marwaha, S., Carter, J.N., Zhou, X., Wheeler, M., Bernstein, J.A., Wang, M., He, P., Zhou, J., Snyder, M., Cong, L., Regev, A., Leskovec, J.: Biomni: A general-purpose biomedical AI agent. *bioRxiv : the preprint server for biology* (2025). <https://doi.org/10.1101/2025.05.30.656746>
17. Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T.T., Moazam, H., Miller, H., Zaharia, M., Potts, C.: Dspy: Compiling declarative language model calls into self-improving pipelines (2024)
18. Le, P.Q., Nauta, M., Nguyen, V.B., Pathak, S., Schlötterer, J., Seifert, C.: Benchmarking eXplainable AI - A Survey on Available Toolkits and Open Challenges. In: Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence. pp. 6665–6673. International Joint Conferences on Artificial Intelligence Organization, Macau, SAR China (Aug 2023). <https://doi.org/10.24963/ijcai.2023/747>
19. Lebo, T., Sahoo, S., McGuinness, D., Belhajjame, K., Cheney, J., Corsar, D., Garijo, D., Soiland-Reyes, S., Zednik, S., Zhao, J.: PROV-O: The PROV Ontology. W3C Recommendation, World Wide Web Consortium, United States (Apr 2013)
20. Liu, G., Solomon, S.: Ai agent observability: Evolving standards and best practices. <https://opentelemetry.io/blog/2025/ai-agent-observability/> (Mar 2025), <https://opentelemetry.io/blog/2025/ai-agent-observability/>, openTelemetry Blog. Accessed: 2026-05-06
21. Liu, R., Luo, B., Li, J., Wang, B., Liu, M., Wu, D., Wang, S., Qin, B.: Ontology-guided reverse thinking makes large language models stronger on knowledge graph question answering. In: Che, W., Nabende, J., Shutova, E., Pilehvar, M.T. (eds.) Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 15269–15284. Association for Computational Linguistics, Vienna, Austria (Jul 2025). <https://doi.org/10.18653/v1/2025.acl-long.741>, <https://aclanthology.org/2025.acl-long.741/>

22. Liu, S., Semnani, S., Triedman, H., Xu, J., Zhao, I.D., Lam, M.: SPINACH: SPARQL-based information navigation for challenging real-world questions. In: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2024*. pp. 15977–16001. Association for Computational Linguistics, Miami, Florida, USA (Nov 2024). <https://doi.org/10.18653/v1/2024.findings-emnlp.938>, <https://aclanthology.org/2024.findings-emnlp.938/>
23. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. p. 4768–4777. NIPS'17, Curran Associates Inc., Red Hook, NY, USA (2017)
24. Luo, H., E, H., Chen, G., Zheng, Y., Wu, X., Guo, Y., Lin, Q., Feng, Y., Kuang, Z., Song, M., Zhu, Y., Tuan, L.A.: Hypergraphrag: Retrieval-augmented generation via hypergraph-structured knowledge representation (2025), <https://arxiv.org/abs/2503.21322>
25. Mavromatis, C., Adeshina, S., Ioannidis, V.N., Han, Z., Zhu, Q., Robinson, I., Thompson, B., Rangwala, H., Karypis, G.: BYOKG-RAG: Multi-strategy graph retrieval for knowledge graph question answering. In: Christodoulopoulos, C., Chakraborty, T., Rose, C., Peng, V. (eds.) *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. pp. 27869–27886. Association for Computational Linguistics, Suzhou, China (Nov 2025). <https://doi.org/10.18653/v1/2025.emnlp-main.1417>, <https://aclanthology.org/2025.emnlp-main.1417/>
26. de Oliveira, R.L., Duarte, J.C., de Faria Cordeiro, K.: Machine learning model explainability supported by data explainability: a provenance-based approach. *Journal of Information and Data Management* **15**(1), 93–102 (Feb 2024). <https://doi.org/10.5753/jidm.2024.3337>, <https://journals-sol.sbc.org.br/index.php/jidm/article/view/3337>
27. O'Sullivan, K., Markovic, M., Dymiter, J., Scheliga, B., Odo, C., Wilde, K.: Semi-automated data provenance tracking for transparent data production and linkage to enhance auditing and quality assurance in Trusted Research Environments **10**(2). <https://doi.org/10.23889/ijpds.v10i2.2464>, <https://ijpds.org/article/view/2464>
28. Pan, X., de Boer, V., van Ossenbruggen, J.: FIRESPARQL: A LLM-based framework for SPARQL query generation over scholarly knowledge graphs (2025), <https://arxiv.org/abs/2508.10467>
29. Procko, T.T., Vonderhaar, L., Ochoa, O.: A survey of machine learning lifecycle provenance: Models, approaches and tools . <https://doi.org/10.2139/ssrn.5682822>, <http://dx.doi.org/10.2139/ssrn.5682822>
30. Ribeiro, M.T., Singh, S., Guestrin, C.: "why should i trust you?": Explaining the predictions of any classifier. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. p. 1135–1144. KDD '16, Association for Computing Machinery, New York, NY, USA (2016). <https://doi.org/10.1145/2939672.2939778>, <https://doi.org/10.1145/2939672.2939778>
31. Schlegel, M., Sattler, K.U.: Capturing end-to-end provenance for machine learning pipelines. *Information Systems* **132**, 102495 (2025). <https://doi.org/https://doi.org/10.1016/j.is.2024.102495>, <https://www.sciencedirect.com/science/article/pii/S0306437924001534>
32. Smeros, P., Emonet, V., Wang, R., Sima, A.C., de Farias, T.M.: SPARQL-LLM: Real-time SPARQL query generation from natural language questions (2025), <https://arxiv.org/abs/2512.14277>

33. Taghzouti, Y., Michel, F., Jiang, T., Nothias, L.F., Gandon, F.: Q2Forge: Mining Competency Questions and SPARQL Queries for Question-Answering Over Knowledge Graphs
34. Targino, N., Souza, D., Salgado, A.C.: An approach to associate provenance from ml pipelines with transparency and accountability. In: 2024 15th International Conference on Information, Intelligence, Systems & Applications (IISA). pp. 1–8 (2024). <https://doi.org/10.1109/IISA62523.2024.10786622>
35. Walter, S., Bast, H.: Grasp: Generic reasoning and sparql generation across knowledge graphs. In: Garijo, D., Kirrane, S., Salatino, A., Shimizu, C., Acosta, M., Nuzzolese, A.G., Ferrada, S., Soulard, T., Kozaki, K., Takeda, H., Gentile, A.L. (eds.) *The Semantic Web – ISWC 2025*. pp. 271–289. Springer Nature Switzerland, Cham (2026)
36. Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., Ji, H.: Executable code actions elicit better llm agents. In: *Proceedings of the 41st International Conference on Machine Learning. ICML’24, JMLR.org* (2024)
37. Wang, X., Yin, M., Koh, E., Dogan, M.D.: Xagen: An explainability tool for identifying and correcting failures in multi-agent workflows. In: *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems. CHI EA ’26*, Association for Computing Machinery, New York, NY, USA (2026). <https://doi.org/10.1145/3772363.3798531>, <https://doi.org/10.1145/3772363.3798531>

A Report Example

Fig. 5: Example report provided by the system.

User Question: Is the function with a description as "Generates a system prompt" made by using Large Language Model output?

KG Question: What are all the objects of class workflow:Generative_Task with a given value for the relation/prop dc:description?

Result Entities:

ChatBS-NexGen:system_prompt_generator

ChatBS-NexGen:system_prompt_generator[provone:Program]

provone:hasInPort -> ChatBS-NexGen:system_prompt_generator-weight
[provone:Port](Input port for weigh ...)

dc:description -> Generates a system prompt based on the provided
dataset structure and user input.@en^<xsd:string>

...

Rewritten Question: Given the current evidence, Focus on: The answer does not provide any explicit literal values. To satisfy the requirement, the answer should include specific literals from the dataset-such as the dc:description value for

ChatBS-NexGen:system_prompt_generator-and any relevant SIO_000202 triple indicating a link to a Large Language Model Output instance. Without these literal details.

KG Question: What type of output does this AI task produce?

Result Entities:

ChatBS-NexGen:LLM_Output-system_prompt_generator

ChatBS-NexGen:LLM_Output-system_prompt_generator

[workflow:Large_Language_Model_Output]

sio:SIO_000202 -> ChatBS-NexGen:system_prompt_generator
[provone:Program](System Prompt Templa ...)

sio:SIO_000232 -> ChatBS-NexGen:LLM-system_prompt_generator
[workflow:Large_Language_Model](System Prompt Templa ...)

Summary Answer:

The function identified as **ChatBS-NexGen:system_prompt_generator** has the literal description

'''

dc:description = "Generates a system prompt based on the provided
dataset structure and user input."

'''

In the provenance graph, this program is linked to an output that is explicitly typed as a Large Language Model Output. The triple

'''

ChatBS-NexGen:LLM_Output-system_prompt_generator sio:SIO_000202

ChatBS-NexGen:system_prompt_generator

'''

indicates that the LLM output was produced *by* (i.e., generated from) the program **system_prompt_generator**. Because the output entity is typed as 'workflow:Large_Language_Model_Output', we can conclude that the function's result-i.e., the system prompt-is indeed created using a Large Language Model. Thus, **yes**, the function with the description "Generates a system prompt" is made by using Large Language Model output.

B Example stepwise execution

Fig. 6: Example of iterative, evidence-grounded retrieval. The system executes *SQs* over the workflow graph, using retrieved entities to refine the question and accumulate evidence across steps, enabling a grounded explanation.

```
User Question: Was the input to the "Input port for text" in the
execution "http://testwebsite/testProgram#id_20260420105703_307"
generated by a generative task?

Step 0:

### Question:
Was the input to the "Input port for text" in the execution
"http://testwebsite/testProgram#id_20260420105703_307" generated by
a Generative Task?

### SQ:
What the attributes of the object?

### Entities:
http://testwebsite/testProgram#id_20260420105703_307[provone:Execution]
  dcterms:identifier -> information_extraction_1_1@en^^<xsd:string>
  prov:used -> ChatBS-NexGen:Data-id_20260420105659_338-generated_answer
               [provone:Data](generated_answer@en^ ...)
  ...

Step 1:

### Question:
Does the data entity 'ChatBS-NexGen:Data-id_20260420105659_338-
generated_answer' have a 'prov:qualifiedGeneration' that links it to a
generation event produced by a 'workflow:Generative_Task', and if so,
what is the URI of that generation event?

### SQ:
What execution produced a given data item?

### Entities:
ChatBS-NexGen:id_20260420105659_302[provone:Execution]
  dcterms:identifier -> llm_chat_1_1@en^^<xsd:string>
  sio:SIO_000369 -> ChatBS-NexGen:Generative_Task-id_20260420105659_302
                   [workflow:Generative_Task](LLM Generates an tex ...)
  ...
```

Table 4: Synthetic question generation metadata for ChatBS-NexGen and Biomni. Path length is measured as the number of class nodes in the schema path.

Dataset	Schema Traversal		SQ Generation		Path Length
	Classes	Raw Paths	Raw SQs	Filtered SQs	Avg.
ChatBS-NexGen	14	385	314	252	4.03
Biomni	14	385	270	208	4.03

C SQ Information

C.1 SQ Library Metadata

To characterize the coverage of the generated *SQ* libraries, we report the schema traversal and filtering statistics for both workflow graphs. This metadata addresses the need to make the *SQ* generation process more transparent by showing how many ontology-derived paths were explored, how many candidate *SQs* were produced, and how many remained after graph-specific filtering. As shown in Table 4, both datasets traverse the same ontology schema, covering 14 classes and 385 raw schema paths. However, the final number of retained *SQs* differs across datasets because the filtering step removes candidate queries that do not return results over the corresponding workflow graph. ChatBS-NexGen retains 252 of 314 generated *SQs*, while Biomni retains 208 of 270 generated *SQs*.

C.2 SQ VS evaluation questions

The evaluation questions were not used during *SQ* generation, *SQ* selection prompt construction, or retrieval-pipeline development. Although both *SQs* and evaluation questions are grounded in the same workflow ontology, the evaluation SPARQL queries were manually authored after *SQ* library construction and were used only to derive ground-truth answers and entity sets (available at https://anonymous.4open.science/r/LLM-Workflow-Explorer-040D/evaluations/biomni-base/ground_truth.ipynb). The *SQ* natural-language questions were not inserted into the evaluation set. Thus, the evaluation measures whether the system can select and compose *SQs* to answer held-out natural-language questions, rather than whether it can reproduce known *SQ* templates.

C.3 SQ Example

This example retrieves the execution instances responsible for generating data that was used as input to a Large Language Model within the workflow.

D Turtle File examples

The following example shows a Turtle serialization snippet for an LLM-based workflow step in ChatBS-NexGen. In this execution, an LLM uses a system prompt

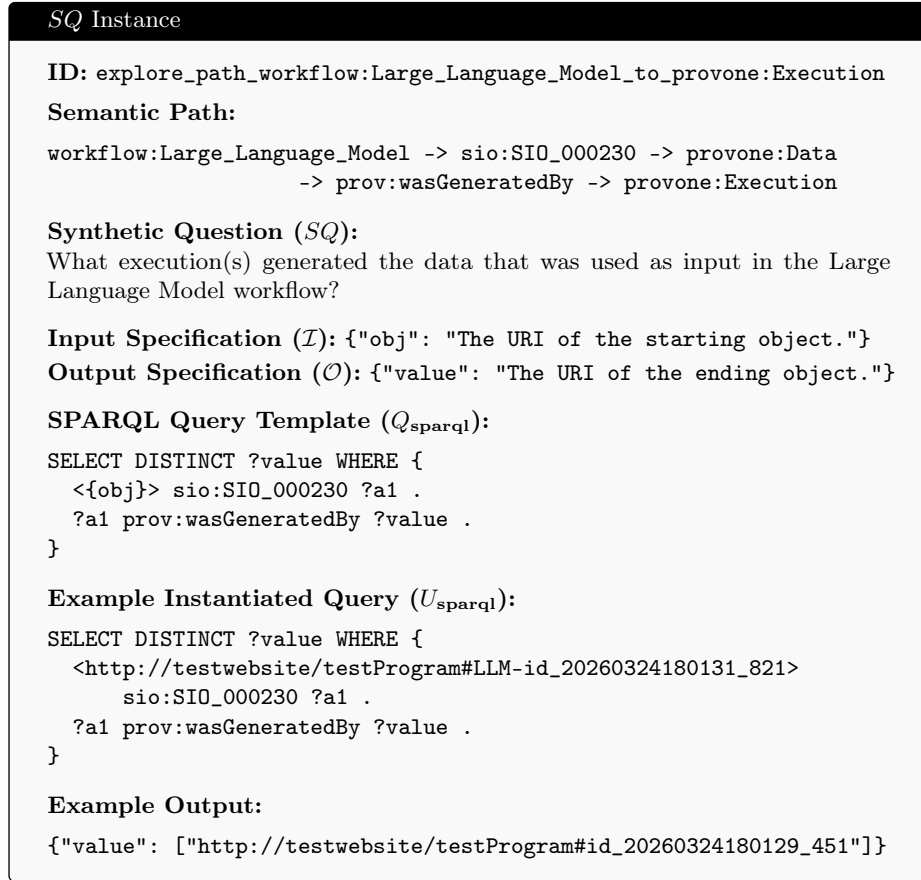


Fig. 7: Example synthetic question instance generated from an ontology path. The instance specifies the semantic path, natural-language question, input/output specification, SPARQL query template, instantiated query, and example output.

that encodes the patient's profile and health indicators, together with a user prompt requesting light, energy-boosting food recommendations before exercise, to generate a dietary recommendation for the patient. The LLM invocation is represented as a generative task that consumes the system and user prompts and produces the generated recommendation. The snippet illustrates how the workflow graph records the model used, input artifacts, generated output, and the provenance relation linking the output to the execution that produced it.

E Evaluation Prompts

Description: Figures 9 and 10 show the structured prompts used for LLM-as-a-judge evaluation. The first assigns normalized scores for *completeness*, *faithfulness*, and *relevance*, while the second performs pairwise comparison for win-rate analysis based on faithfulness and understandability.

F Baseline Adaptation Details

The ChatBS-NexGen workflow graph contains 895 RDF triples over 177 URI subjects, while the Biomni workflow graph contains 2,022 RDF triples over 230 URI subjects. Unless otherwise noted, labels are taken from `rdfs:label` and `skos:prefLabel`; descriptions from `dc:description` and `prov:definition`; and identifiers from `dcterms:identifier`.

The baselines differ in their native input requirements: GRASP operates over a SPARQL endpoint, HippoRAG expects passages and OpenIE-style facts, HyperGRAG expects chunks, entities, and hyperedges, and VSB retrieves over embedded text segments. To ensure a fair comparison, each baseline is constructed from the same underlying workflow KG, receives the same natural-language questions, and is not given any external corpus. The only difference is the representation used to adapt the KG to each baseline’s expected input format. All local LLM calls use `gpt-oss-20b` through the LM Studio. vector-based methods use `text-embedding-bge-large-en-v1.5` unless the baseline requires its own embedding index.

For text-based and graph-RAG baselines, we enumerate every URI subject in the workflow KG and construct one object-level representation per subject. Each representation includes a CURIE, a human-readable label derived from labels or identifiers, `rdf:type` values, and non-type outgoing triples rendered as property lines containing the predicate CURIE, predicate label, object CURIE when applicable, and object label or literal. VSB embeds these object descriptions directly. HippoRAG receives the same descriptions as passages, along with an injected OpenIE file whose `extracted_entities` and `extracted_triples` are produced mechanically from the RDF triples rather than generated by an LLM. HyperGRAG receives the same descriptions as chunks; non-type RDF triples are converted into hyperedge-like records, while URI objects and literals are represented as entity nodes. GRASP is adapted differently: instead of receiving a flattened corpus, it queries the same workflow KG through a Fuseki SPARQL endpoint and uses entity/property indices derived from labels, aliases, comments, domain/range information, observed usage, and example values where available. Table 5 summarizes the native representation, graph access, and ontology/schema access provided to each baseline.

F.1 Latency and Call Count Analysis

Table 6 reports the average latency and call counts for each method across the ChatBS-NexGen and Biomni benchmarks. Vector-based and graph-RAG baselines require fewer LLM calls and therefore achieve lower latency, while GRASP incurs additional cost from iterative SPARQL generation and execution. Our method has the highest latency, particularly on Biomni, because it performs multiple LLM-guided steps for SQ selection, parameter grounding, answer validation, and refinement. This reflects a trade-off between efficiency and explanation quality: as shown in the main evaluation results, the additional calls enable stronger grounding and higher-quality explanations, but reduce suitability for latency-sensitive settings.

Table 6: Dataset-specific average latency (s) and call counts for each baseline and our method. SPARQL calls denote explicit query-bearing tool calls for GRASP. Dashes indicate metrics not available for the corresponding method.

Method	ChatBS-NexGen			Biomni		
	Latency	LLM	SPARQL	Latency	LLM	SPARQL
VSB	2.94	1.00	-	3.18	1.00	-
GRASP	15.49	7.79	5.79	10.57	5.06	1.63
HippoRAG	4.59	1.00	-	4.83	1.00	-
HyperGRAG	12.67	1.74	-	14.11	1.91	-
Ours	39.98	6.95	2.59	146.06	9.32	2.74

F.2 Developer Workload

Table 7 summarizes the effort needed to add provenance capture to the two systems. Both systems required a small number of high-level annotation calls. ChatBS-NexGen required wrapping a fixed pipeline, while Biomni required more adapter code to capture dynamic agent iterations, code execution, critic steps, and intermediate artifacts. Instrumentation time was not recorded.

Table 7: Instrumentation overhead for ChatBS-NexGen and Biomni. Files denotes modified files, Calls denotes annotation calls, LOC denotes added lines of code, Time denotes instrumentation time, and Triples denotes produced RDF triples. - indicates not recorded.

System	Files	Calls	LOC	Time	Triples
ChatBS-NexGen	5	30	870	-	895
Biomni	3	31	1,172	-	2,022

Fig. 8: Example Turtle serialization of an LLM-based workflow step. The excerpt shows how a generative task, LLM invocation, input artifacts, and output artifact are represented in the workflow graph.

```

@prefix ex:      <http://testwebsite/testProgram#> .
@prefix prov:    <http://www.w3.org/ns/prov#> .
@prefix provone: <http://purl.dataone.org/provone/2015/01/15/ontology#> .
@prefix sio:     <http://semanticscience.org/resource/> .
@prefix workflow: <http://www.semanticweb.org/acer/ontologies/2026/1/Workflow/> .
@prefix rdfs:    <http://www.w3.org/2000/01/rdf-schema#> .
@prefix dc:      <http://purl.org/dc/elements/1.1/> .

ex:Generative_Task-id_20260420105659_302
  a workflow:Generative_Task ;
  rdfs:label "LLM text generation task" ;
  dc:description "Generates a dietary recommendation using an LLM." ;
  prov:used ex:LLM-id_20260420105659_302 ;
  sio:SIO_000313 ex:id_20260420105659_302 .

ex:LLM-id_20260420105659_302
  a workflow:Large_Language_Models ;
  workflow:llm_model "gpt-4o" ;
  sio:SIO_000230 ex:Data-id_20260420105652_456-system_prompt ;
  sio:SIO_000230 ex:Data-id_20260420105659_811-user_prompt ;
  sio:SIO_000229 ex:LLM_Output-id_20260420105659_302 .

ex:Data-id_20260420105652_456-system_prompt
  a provone:Data ;
  rdfs:label "system_prompt" ;
  prov:value "You are a health care assistant..." .

ex:Data-id_20260420105659_811-user_prompt
  a provone:Data ;
  rdfs:label "user_prompt" ;
  prov:value "I need suggestions for light and energy-boosting foods..." .

ex:Data-id_20260420105659_338-generated_answer
  a provone:Data ;
  rdfs:label "generated_answer" ;
  prov:value "To fuel your body effectively before a gym workout..." ;
  prov:wasGeneratedBy ex:id_20260420105659_302 .

ex:Generation-id_20260420105659_282-generated_answer
  a prov:Generation ;
  provone:hadEntity ex:Data-id_20260420105659_338-generated_answer ;
  provone:hadOutPort ex:llm_chat-generated_answer .

```

AnswerQualityJudgeSignature (DSPy Prompt)

```
class AnswerQualityJudgeSignature(dspy.Signature):
    """Score answer completeness, accuracy, and relevance against
    the ground truth."""

    question: str = dspy.InputField()
    ground_truth_answer: str = dspy.InputField()
    model_answer: str = dspy.InputField()

    completeness: float = dspy.OutputField(desc="How much of the
        ground truth is covered, from 0 to 1.")
    faithfulness: float = dspy.OutputField(desc="How faithful the
        answer is to the ground truth, from 0 to 1.")
    relevance: float = dspy.OutputField(desc="How directly the
        answer addresses the question, from 0 to 1.")
```

Fig. 9: DSPy signature used for absolute LLM-as-a-judge scoring of generated explanations.

PairwiseAnswerWinrateSignature (DSPy Prompt)

```
class PairwiseAnswerWinrateSignature(dspy.Signature):
    """You are an evaluator and you are given outputs of two
    explanation generation systems for AI systems. Your job is
    to identify which method provides a better explanation to
    the asked question. A human-curated answer is also provided.

    Choose which answer is more understandable by a lay user.
    Be cautious whether the answer is faithful to the human-curated
    answer."""

    question: str = dspy.InputField()
    ground_truth_answer: str = dspy.InputField()
    method_a: str = dspy.InputField()
    answer_a: str = dspy.InputField()
    method_b: str = dspy.InputField()
    answer_b: str = dspy.InputField()
    winner: str = dspy.OutputField(
        desc="Return exactly one of: method_a, method_b, tie."
    )
```

Fig. 10: DSPy signature used for pairwise LLM-as-a-judge win-rate evaluation.

Table 5: Native representation and graph access used by each baseline. “Same triples” indicates whether the baseline was constructed from the same execution KG, although the retrieval-time answer context may still be a subset.

Method	Native input representation	Same triples as ours?	Ontology/schema access
GWB	An in-memory <code>rdflib</code> graph converted into entity, predicate, triple, and execution records. The LLM only rewrites an already-grounded symbolic draft.	Yes. Symbolic indices are built from the full graph.	Yes. Used to name predicates and classes.
VSb	One text segment per URI subject, listing the subject CURIE, label, RDF types, and outgoing properties with object labels or literals.	Yes. Segments are constructed from the full graph.	Partial. Ontology labels are used for predicates, but RDF domain/range semantics are not reasoned over after flattening.
GRASP	The workflow graph is exposed as a SPARQL endpoint. GRASP searches entity/property indices and iteratively generates and executes SPARQL.	Yes. The SPARQL endpoint is loaded with the same workflow graph.	Partial. GRASP receives prefixes and queryable labels, comments, domain/range, and type information from the endpoint/index. It does not receive our generated SQ Library.
HippoRAG	The KG is converted to one passage per URI subject plus an injected OpenIE file. ChatBS-NexGen produces 177 passages and Biomni produces 230 passages; the injected OpenIE contains 895 ChatBS-NexGen triples and 2,022 Biomni triples.	Yes. Triples are mechanically flattened into passage text and injected OpenIE triples.	Partial. Ontology labels are used during conversion, but HippoRAG receives passages/OpenIE facts rather than the formal ontology schema.
HyperGRAG	The same per-subject KG passages are converted into HyperGraphRAG chunks, entities, and hyperedges. ChatBS-NexGen yields 177 chunks, 345 entities, and 718 hyperedges; Biomni yields 230 chunks, 1,169 entities, and 1,792 hyperedges.	Mostly. Non- <code>rdf:type</code> outgoing triples become hyperedges; type triples become entity metadata and remain in chunk text.	Partial. Ontology labels are used during conversion, but HyperGRAG receives a flattened hypergraph rather than RDF/OWL semantics.